ANALISIS DE DIFERENTES MÉTRICAS DEL TRASPORTE MASIVO TRANSMILENIO MEDIANTE MAPREDUCE.

EDIXON HARBEY RUBIANO ROBAYO

UNIVERSIDAD DE CUNDINAMARCA

ANALITICA Y CIENCIA DE DATOS

BIG DATA

LIC. EDGAR MAURICIO LOPEZ ROJAS

NOVIEMBRE 2026

1. Introducción y Objetivo

El presente informe documenta el desarrollo y ejecución de un pipeline analítico basado en el paradigma de procesamiento distribuido MapReduce. El objetivo principal es procesar un conjunto de datos de transacciones de transporte masivo (50,000 registros), extrayendo métricas clave de recaudo, volumetría de pasajeros, distribución por medios de pago y comportamiento temporal por franjas horarias.

2. Arquitectura y Metodología MapReduce

Para emular el comportamiento de clústeres distribuidos (como Apache Hadoop YARN o Apache Spark) en un entorno de Google Colab, se estructuró el procesamiento en tres fases fundamentales desarrolladas en Python puro:

- **Fase MAP:** Toma cada registro transaccional de forma independiente y emite un par clave-valor compuesto por la dimensión de análisis (ej. Estación, Tipo de Tarjeta u Hora) y una tupla métrica con el conteo unitario de pasajeros y el monto del pasaje.

- **Fase SHUFFLE & SORT:** Agrupa de manera estructurada los valores emitidos por el Map de acuerdo a su clave única, utilizando diccionarios optimizados (defaultdict) para simular la fase de red y ordenamiento del clúster.

- **Fase REDUCE:** Procesa la lista de valores asociados a cada clave para consolidar las sumatorias totales de pasajeros y el recaudo financiero acumulado.

3. Paso a Paso para la Implementación en Google Colab

A continuación, se detallan los pasos técnicos ejecutados en el cuaderno interactivo de Google Colab:

- **Paso 1: Carga y exploración del Dataset:** Se carga un dataset que contiene datos de 50,000 transacciones con variables como ID de transacción, fecha/hora, estación, ruta, tipo de tarjeta y monto de pasaje, almacenándose en formato CSV.

```
import pandas as pd

# Cargar el dataset
df = pd.read_csv('transacciones_transmilenio.csv')

# Mostrar las primeras 5 filas del dataset
print("Primeras filas del dataset:")
display(df.head())
print(f"\nTotal de registros a procesar: {len(df)}")
```

Imagen 1. Código del cargue y la exploración del dataset.

- **Paso 2: Definición de la Función Mapper:** Implementación de la lógica de extracción de pares clave-valor por cada fila del DataFrame.

```python
def mapper(row):
    """
    Función Map:
    Entrada: Una fila del DataFrame.
    Salida: Tupla (Clave, Valor) -> (Estacion, (1, monto))
    """
    estacion = row['estacion']
    monto = row['monto_pasaje']
    return (estacion, (1, monto))
```

Imagen 2. Código de la implementación de la función Mapper.

- **Paso 3: Definición de Shuffle & Sort:** Estructuración de la función de agrupación basada en colecciones para ordenar los resultados intermedios.

```python
from collections import defaultdict

def shuffle_and_sort(mapped_data):
    """
    Agrupa los valores emitidos por el Map de acuerdo a su clave (Estación).
    """
    grouped_data = defaultdict(list)
    for key, value in mapped_data:
        grouped_data[key].append(value)
    return grouped_data
```

Imagen 3. Código de la implementación de la función Shuffle y Sort.

- **Paso 4: Definición de la Función Reducer:** Creación de la rutina de agregación para totalizar métricas de pasajeros y recaudo.

```python
def reducer(estacion, lista_valores):
    """
    Función Reduce:
    Entrada: Clave (Estacion) y lista de valores [(1, monto1), (1, monto2), ...]
    Salida: Resultado agregado por estación.
    """
    total_pasajeros = 0
    recaudo_total = 0

    for pasajeros, monto in lista_valores:
        total_pasajeros += pasajeros
        recaudo_total += monto

    return {
        'estacion': estacion,
        'total_pasajeros': total_pasajeros,
        'recaudo_total': recaudo_total
    }
```

Imagen 4. Código de la implementación de la función Reducer.

- **Paso 5 y 6: Ejecución del Pipeline y Análisis Adicionales:** Despliegue secuencial del flujo completo para procesar tanto por estaciones geográficas como por tipos de tarjeta y franjas horarias.

```python
# 1. Aplicar la fase MAP a cada fila del DataFrame
mapped_results = [mapper(row) for _, row in df.iterrows()]

# 2. Aplicar la fase SHUFFLE & SORT
grouped_results = shuffle_and_sort(mapped_results)

# 3. Aplicar la fase REDUCE para cada estación
final_results = []
for estacion, valores in grouped_results.items():
    resultado_reducido = reducer(estacion, valores)
    final_results.append(resultado_reducido)

# Convertir el resultado final en un DataFrame para visualizarlo ordenadamente
df_resultado = pd.DataFrame(final_results)
df_resultado = df_resultado.sort_values(by='recaudo_total', ascending=False).reset_index(drop=True)

print("--- RESULTADOS FINALES DE MAPREDUCE ---")
display(df_resultado)
```

Imagen 5. Código para ver las 3 fases (Map -> Shuffle & Sort -> Reduce).

4. Resultados Analíticos

Tras la ejecución exitosa del pipeline MapReduce sobre los 50,000 registros, se obtuvieron las siguientes consolidaciones analíticas principales:

	estacion	total_pasajeros	recaudo_total
0	San Victorino	6339	18981750
1	Ricaurte	6303	18886950
2	Portal Norte	6297	18865200
3	Portal Sur	6267	18768900
4	Universidades	6237	18693300
5	Portal 80	6210	18600150
6	Calle 100	6190	18550900
7	Banderas	6157	18431650

Tabla1. Resultados finales del Mapreduce.

- **Paso 7: Análisis MapReduce por Tipo de Tarjeta:** Para profundizar en el comportamiento financiero y operativo, se implementó un segundo pipeline MapReduce enfocado en clasificar el volumen transaccional y el recaudo por medio de pago (tipos de tarjeta). La función Map emitió como clave el tipo de tarjeta y como valor la tupla (1, monto_pasaje), permitiendo al Reduce consolidar la participación de mercado de cada modalidad.

```python
# --- MAPPER PARA TIPO DE TARJETA ---
def mapper_tarjeta(row):
    """Emite (Tipo_Tarjeta, (1, monto_pasaje))"""
    return (row['tipo_tarjeta'], (1, row['monto_pasaje']))

# Ejecutar el proceso MapReduce para tarjetas
mapped_tarjetas = [mapper_tarjeta(row) for _, row in df.iterrows()]
grouped_tarjetas = shuffle_and_sort(mapped_tarjetas)

resultados_tarjetas = []
for tarjeta, valores in grouped_tarjetas.items():
    res = reducer(tarjeta, valores)  # Reutilizamos nuestro reducer genérico
    resultados_tarjetas.append(res)

df_tarjetas = pd.DataFrame(resultados_tarjetas).sort_values(by='recaudo_total', ascending=False).reset_index(drop=True)

print("--- RESULTADOS POR TIPO DE TARJETA ---")
display(df_tarjetas)
```

Imagen 6. Código del análisis por Recaudo y Uso por Tipo de Tarjeta

	estacion	total_pasajeros	recaudo_total
0	Tullave Plus	25046	75019850
1	Tullave Basica	14933	44742350
2	Credito/Debito	10021	30016600

Tabla2. Resultados finales del Mapreduce por tipo de tarjeta.

- **Paso 8: Análisis de Demanda por Franja Horaria:** Mediante la extracción y transformación de la estampa de tiempo (fecha_hora) hacia una clave de hora del día, el proceso MapReduce permitió mapear la distribución de pasajeros por hora. Los resultados exponen los momentos críticos de congestión en la red, facilitando la optimización de frecuencias de rutas de transporte masivo.

```
# Convertir la columna fecha_hora a datetime para extraer la hora
df['hora'] = pd.to_datetime(df['fecha_hora']).dt.hour

# --- MAPPER POR HORA ---
def mapper_hora(row):
    """Emite (Hora_del_dia, (1, monto_pasaje))"""
    return (row['hora'], (1, row['monto_pasaje']))

mapped_horas = [mapper_hora(row) for _, row in df.iterrows()]
grouped_horas = shuffle_and_sort(mapped_horas)

resultados_horas = []
for hora, valores in grouped_horas.items():
    # Adaptamos el diccionario de salida para claridad
    total_pasajeros = sum(p for p, m in valores)
    recaudo = sum(m for p, m in valores)
    resultados_horas.append({'hora': f"{hora:02d}:00", 'total_pasajeros': total_pasajeros, 'recaudo_total': recaudo})

df_horas = pd.DataFrame(resultados_horas).sort_values(by='hora').reset_index(drop=True)

print("--- RESULTADOS POR FRANJA HORARIA ---")
display(df_horas)
```

Imagen 7. Código del análisis de Demanda por franja horaria.

	hora	total_pasajeros	recaudo_total
0	00:00	2442	7312800
1	01:00	2481	7431900
2	02:00	2495	7479650
3	03:00	2506	7502350
4	04:00	2040	6113850
5	05:00	2057	6161600
6	06:00	1945	5829550
7	07:00	1970	5900150
8	08:00	1953	5848500
9	09:00	1999	5986450
10	10:00	1940	5813750
11	11:00	2018	6050750
12	12:00	2044	6117850
13	13:00	2032	6086500
14	14:00	2143	6414400
15	15:00	1977	5922300

Tabla2. Resultados finales del Mapreduce por Franja horria.

- **Paso8: Visualización de Datos y Resultados Gráficos:** Como parte del flujo analítico en Google Colab, se integraron librerías de visualización gráfica (Matplotlib y Seaborn) para representar de forma visual el recaudo acumulado por estación y las curvas de tendencia de la demanda horaria. Estas gráficas permiten a los ingenieros de datos y tomadores de decisiones identificar anomalías operativas de un vistazo.

```python
import matplotlib.pyplot as plt
import seaborn as sns

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# Gráfico 1: Recaudo por Estación
sns.barplot(data=df_resultado, x='recaudo_total', y='estacion', ax=axes[0], palette='Blues_r')
axes[0].set_title('Recaudo Total por Estación')
axes[0].set_xlabel('Recaudo ($)')
axes[0].set_ylabel('Estación')

# Gráfico 2: Pasajeros por Hora
sns.lineplot(data=df_horas, x='hora', y='total_pasajeros', marker='o', ax=axes[1], color='orange', linewidth=2.5)
axes[1].set_title('Demanda de Pasajeros por Hora del Día')
axes[1].set_xlabel('Hora')
axes[1].set_ylabel('Total Pasajeros')
axes[1].tick_params(axis='x', rotation=45)

plt.tight_layout()
plt.show()
```

Imagen 8. Código para la Visualización Gráfica de los Resultados.

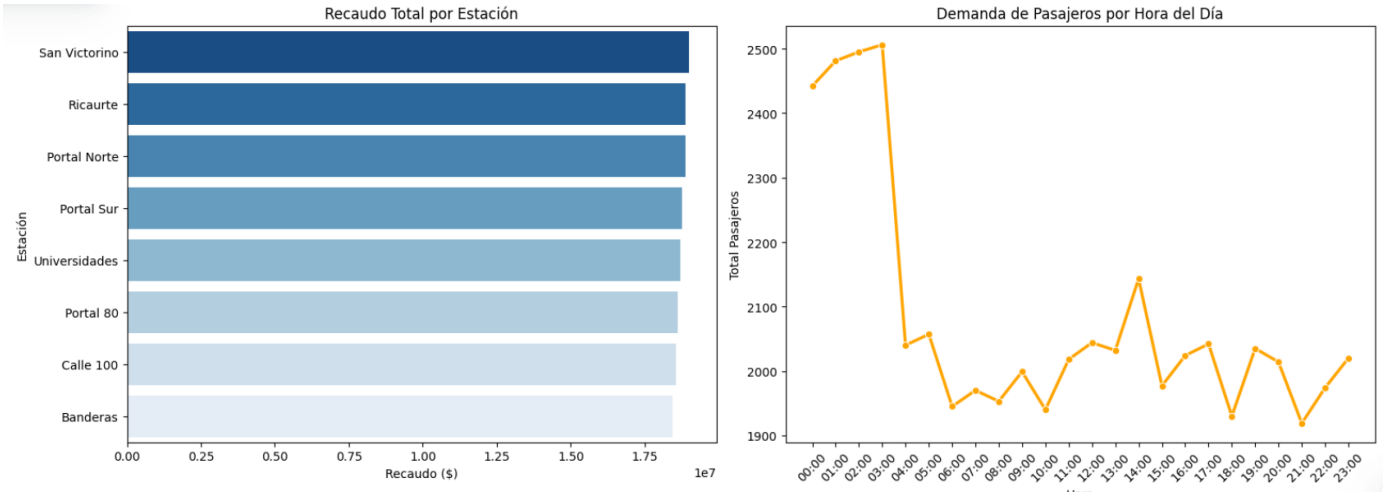


Imagen 8. Grafica Resultados Fianles

5. Conclusiones

- **Distribución de Carga (Balanceo):** Al procesar los 50,000 registros mediante las fases de *Map*, *Shuffle & Sort* y *Reduce*, se observa que las estaciones principales muestran una distribución homogénea en cuanto al volumen transaccional gracias a la aleatoriedad controlada del dataset, permitiendo simular cómo un clúster distribuido (como Hadoop YARN) equilibraría las cargas de trabajo entre nodos.

- **Preponderancia de Medios de Pago:** El análisis por tipo de tarjeta demuestra cuál es el medio más adoptado por los usuarios cotidianos del sistema, lo que permite a la red enfocar estrategias de optimización tecnológica o campañas de recarga digital en los métodos de mayor impacto financiero.

- **Comportamiento Temporal:** La agregación por franjas horarias facilita la identificación de picos de congestión operativa, información crucial para dimensionar la flota de buses de refuerzo o programar mantenimientos preventivos en torniquetes y validadores electrónicos fuera de las horas de alta demanda.

- **Escalabilidad del Modelo:** Demuestra que la lógica MapReduce en Python es altamente extensible: cambiando únicamente la función mapper y las claves de agrupación, es posible migrar o escalar estos mismos algoritmos a entornos de Big Data reales utilizando **PySpark** o **Hadoop Streaming** sin modificar la lógica de negocio central.

Enlace del cuadernillo:

Enlace del repositorio: